

01_chunking_experiment

October 4, 2026

1 Section 1: Experiment Objective

1.1 VisualMind - Chunking Experiment

1.1.1 Objective

Find a good way to split study documents into chunks for the VisualMind RAG pipeline.

1.1.2 Questions this notebook answers

1. What does a study PDF look like after text extraction?
2. How do we clean it?
3. How do chunk size and overlap change the chunks?
4. Which settings keep definitions, formulas, activities and explanations together?

1.1.3 Documents used

- doc1_life_processes.pdf (text-heavy, Biology)
- doc2_electricity.pdf (formulas and numericals, Physics)

1.1.4 Output of this notebook

- data/eval/gold_questions.json (test questions reused in later notebooks)
- data/processed/chunks.jsonl and data/processed/chunking_config.json (used by Experiment 2)

1.1.5 Out of scope

Embeddings, vector search and LLM answers come in later notebooks.

Cells are marked # [Cell N] so you can compare this notebook with yours.

1.2 Section 2: Imports & Configuration

```
[1]: import warnings, logging
warnings.filterwarnings("ignore", category=DeprecationWarning)
logging.getLogger("pypdf").setLevel(logging.ERROR)

import re, json, random
from pathlib import Path
import numpy as np
```

```

import pandas as pd
import tiktoken
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

print("Imports OK")

```

Imports OK

```

[2]: # Paths and documents
PROJECT_ROOT = Path.cwd().parent
DOCUMENT_DIR = PROJECT_ROOT / "data" / "raw" / "documents"
EVAL_DIR      = PROJECT_ROOT / "data" / "eval"
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"

DOC_NAMES = ["doc1_life_processes.pdf", "doc2_electricity.pdf"]

print("Project root :", PROJECT_ROOT)
print("Document dir :", DOCUMENT_DIR)

```

Project root : c:\Users\ASUS\Desktop\VisualMind

Document dir : c:\Users\ASUS\Desktop\VisualMind\data\raw\documents

1.3 Section 3: Locate Documents

```

[3]: missing = [n for n in DOC_NAMES if not (DOCUMENT_DIR / n).exists()]
print("Folder exists:", DOCUMENT_DIR.exists())
for name in DOC_NAMES:
    path = DOCUMENT_DIR / name
    if path.exists():
        print(f"FOUND {name} ({path.stat().st_size / 1024:.0f} KB)")
    else:
        print(f"MISSING {name}")
print("CHECK:", "PASS" if not missing else f"FAIL - missing {missing}")

```

Folder exists: True

FOUND doc1_life_processes.pdf (8811 KB)

FOUND doc2_electricity.pdf (2535 KB)

CHECK: PASS

1.4 Section 4: Load PDF

```

[4]: # Load each PDF page by page
docs = {}
for name in DOC_NAMES:
    loader = PyPDFLoader(str(DOCUMENT_DIR / name))
    docs[name] = loader.load()
    print(f"{name}: {len(docs[name])} pages loaded")

```

```
doc1_life_processes.pdf: 21 pages loaded
doc2_electricity.pdf: 24 pages loaded
```

1.5 Section 5: Inspect Document Structure

```
[5]: # Size of each document and each page
for name, pages in docs.items():
    lengths = [len(p.page_content) for p in pages]
    print(f"\n{name}")
    print("  Pages          :", len(pages))
    print("  Total chars     :", sum(lengths))
    print("  Min chars/page  :", min(lengths))
    print("  Max chars/page  :", max(lengths))
    print("  Avg chars/page  :", round(sum(lengths) / len(lengths)))
    short_pages = [i for i, n in enumerate(lengths) if n < 200]
    print("  Pages with <200 chars (index):", short_pages)    # a scanned/image
↳page would show up here
```

```
doc1_life_processes.pdf
  Pages          : 21
  Total chars     : 52954
  Min chars/page  : 1964
  Max chars/page  : 3169
  Avg chars/page  : 2522
  Pages with <200 chars (index): []
```

```
doc2_electricity.pdf
  Pages          : 24
  Total chars     : 45646
  Min chars/page  : 942
  Max chars/page  : 2870
  Avg chars/page  : 1902
  Pages with <200 chars (index): []
```

1.6 Section 6: Inspect Raw Text

```
[6]: # Raw text, Biology page 3
text = docs["doc1_life_processes.pdf"][3].page_content
print(text[:1500])
print("-" * 60)
print(repr(text[:600]))    # repr shows the hidden \n characters
```

Science82

Let us now see what actually happens during the process of photosynthesis. The following events occur during this process -
Figure 5.2Figure 5.2Figure 5.2Figure 5.2Figure 5.2
Variegated leaf (a) before

and (b) after starch test

Figure 5.1Figure 5.1Figure 5.1Figure 5.1Figure 5.1

Cross-section of a leaf

Activity 5.1Activity 5.1Activity 5.1Activity 5.1Activity 5.1

/square6Take a potted plant with variegated leaves - for example, money plant or crotons.

/square6Keep the plant in a dark room for three days so that all the starch gets used up.

/square6Now keep the plant in sunlight for about six hours.

/square6Pluck a leaf from the plant. Mark the green areas in it and trace them on a sheet of paper .

/square6Dip the leaf in boiling water for a few minutes.

/square6After this, immerse it in a beaker containing alcohol.

/square6Carefully place the above beaker in a water -bath and heat till the alcohol begins to boil.

/square6What happens to the colour of the leaf? What is the colour of the solution?

/square6Now dip the leaf in a dilute solution of iodine for a few minutes.

/square6Take out the leaf and rinse off the iodine solution.

/square6Observe the colour of the leaf and compare this with the tracing of the leaf done in the beginning (Fig. 5.2).

/square6What can you conclude about the presence of starch in various areas of the leaf?

(i) Absorption of light energy by chlorophyll.

(ii) Conversion of light energy to chemical energy and splitting of water molecules into h

'Science82\nLet us now see what actually happens during the process of\nphotosynthesis. The following events occur during this process -\nFigure 5.2Figure 5.2Figure 5.2Figure 5.2\nVariegated leaf (a) before\nand (b) after starch test\nFigure 5.1Figure 5.1Figure 5.1Figure 5.1\nCross-section of a leaf\nActivity 5.1Activity 5.1Activity 5.1Activity 5.1\n/square6Take a potted plant with variegated leaves - for example, money plant\nor crotons.\n/square6Keep the plant in a dark room for three days so that all the starch\ngets used up.\n/square6Now keep the plant in sunlight for abo'

```
[7]: # Raw text, Physics page 5
text = docs["doc2_electricity.pdf"][5].page_content
print(text[:1500])
print("-" * 60)
print(repr(text[:600]))
```

Science176

Figure 11.3Figure 11.3Figure 11.3Figure 11.3Figure 11.3

V-I graph for a nichrome wire. A straight line plot shows that as the

current through a wire increases, the potential difference across the wire increases linearly - this is Ohm's law.

Activity 11.2Activity 11.2Activity 11.2Activity 11.2Activity 11.2

/square6Take a nichrome wire, a torch bulb, a 10 W bulb and an ammeter (0 - 5 A range), a plug

key and some connecting wires.

/square6Set up the circuit by connecting four dry cells of 1.5 V each in series with the ammeter

leaving a gap XY in the circuit, as shown in Fig. 11.4.

In this Activity, you will find that approximately the same value for V/I is obtained in each case. Thus the $V-I$ graph is a straight line that passes through the origin of the graph, as shown in Fig. 11.3. Thus, V/I is a constant ratio.

In 1827, a German physicist Georg Simon Ohm (1787-1854) found out the relationship between the current I , flowing in a metallic wire and the potential difference across its terminals. The potential difference, V , across the ends of a given metallic wire in an electric circuit is directly proportional to the current flowing through it, provided its temperature remains the same. This is called Ohm's law. In other words -

$V \propto I$ (11.4)

or $V/I = \text{constant}$

$= R$

or $V = IR$ (11.5)

In Eq. (11.4), R is a constant for the given metallic wire at a given temperature and is called its resistance. It is the property of a conductor to resist the flow of charges through it. I

'Science176\nFigure 11.3Figure 11.3Figure 11.3Figure 11.3Figure 11.3\nV-I graph for a nichrome wire. A\nstraight line plot shows that as the\ncurrent through a wire increases, the\npotential difference across the wire\nincreases linearly - this is Ohm's law.\nActivity 11.2Activity 11.2Activity 11.2Activity 11.2Activity 11.2\n/square6Take a nichrome wire, a torch bulb, a 10 W bulb and an ammeter (0 - 5 A range), a plug\nkey and some connecting wires.\n/square6Set up the circuit by connecting four dry cells of 1.5 V each in series with the ammeter\nleaving a gap XY in the circuit, as shown in Fig. 11.4.\nIn '

[8]: # Metadata (only the useful keys)

```
keep = ["title", "page", "page_label", "total_pages", "creator"]
for name, pages in docs.items():
    print(f"\n{name}")
    print("  First page:", {k: pages[0].metadata.get(k) for k in keep})
    print("  Last page :", {k: pages[-1].metadata.get(k) for k in keep})
```

doc1_life_processes.pdf

First page: {'title': 'CHAP 5.pmd', 'page': 0, 'page_label': '1',
'total_pages': 21, 'creator': 'PageMaker 7.0'}

Last page : {'title': 'CHAP 5.pmd', 'page': 20, 'page_label': '21',
'total_pages': 21, 'creator': 'PageMaker 7.0'}

doc2_electricity.pdf

First page: {'title': 'CHAP 11.pmd', 'page': 0, 'page_label': '1',
'total_pages': 24, 'creator': 'PageMaker 7.0'}

Last page : {'title': 'CHAP 11.pmd', 'page': 23, 'page_label': '24',
'total_pages': 24, 'creator': 'PageMaker 7.0'}

1.7 Section 7: Quantify the Text Problems

```
[9]: # Problems in the RAW text (before cleaning)
for name, pages in docs.items():
    text = "\n".join(p.page_content for p in pages)
    lines = [l for l in text.split("\n") if l.strip()]
    print(name)
    print(" Running headers (Science82 ...) :", len(re.findall(r"^\s*(?:
↪Science|Life Processes|Electricity)\s?\d{1,3}", text, flags=re.M)))
    print(" Repeated captions (Figure 5.2x5) :", len(re.findall(r"((?:
↪Figure|Activity|Table)\s?\d+\.\d+)\d+", text)))
    print(" Broken '/square' bullets          :", text.count("/square"))
    print(" Lines not ending . ? ! :          :", sum(not l.strip().endswith(("
↪", "?", "!", ":")) for l in lines), "of", len(lines))
```

doc1_life_processes.pdf

Running headers (Science82 ...) : 21
Repeated captions (Figure 5.2x5) : 19
Broken '/square' bullets : 64
Lines not ending . ? ! : : 780 of 1013

doc2_electricity.pdf

Running headers (Science82 ...) : 24
Repeated captions (Figure 5.2x5) : 18
Broken '/square' bullets : 47
Lines not ending . ? ! : : 730 of 930

1.8 Section 8: Baseline Without Chunking

```
[10]: # What if we send the whole document to the LLM?
for name, pages in docs.items():
    full_text = "\n".join(p.page_content for p in pages)
    print(name)
    print(" Whole document chars          :", len(full_text))
    print(" Approx tokens (chars / 4) :", len(full_text) // 4)
    print(" Page-level units          :", len(pages))
```

```
doc1_life_processes.pdf
  Whole document chars      : 52974
  Approx tokens (chars / 4) : 13243
  Page-level units         : 21
doc2_electricity.pdf
  Whole document chars      : 45669
  Approx tokens (chars / 4) : 11417
  Page-level units         : 24
```

```
[11]: # What if we send whole pages that mention the topic?
def pages_mentioning(pages, keyword):
    return [i for i, p in enumerate(pages) if keyword.lower() in p.page_content.
            ↪lower()]

tests = {"doc1_life_processes.pdf": "photosynthesis", "doc2_electricity.pdf":
        ↪"Ohm"}

for name, keyword in tests.items():
    pages = docs[name]
    hits = pages_mentioning(pages, keyword)
    chars = sum(len(pages[i].page_content) for i in hits)
    print(f"\n{name} keyword='{keyword}'")
    print("  Pages that mention it :", hits)
    print("  Chars we'd send to LLM:", chars)
```

```
doc1_life_processes.pdf keyword='photosynthesis'
  Pages that mention it : [2, 3, 4, 8, 10, 15, 16, 17, 19]
  Chars we'd send to LLM: 23429
```

```
doc2_electricity.pdf keyword='Ohm'
  Pages that mention it : [4, 5, 7, 9, 10, 12, 13, 14, 15, 16, 18, 21]
  Chars we'd send to LLM: 21904
```

1.9 Section 9: Text Cleaning

What `clean_text` fixes:

Problem in raw text	Fix
Science82 running page header	removed
Figure 5.2Figure 5.2... repeated captions	collapsed to one
/square6 bullets, Reprint 2022-23 footer	- bullet / removed
Every line break is a <code>\n</code>	paragraphs rebuilt
Overprinted headings (<code>HEADINGHEADING...</code>)	collapsed
Heading separated from its paragraph	kept together

```

[12]: # clean_text
def glue_blocks(paragraphs):
    """Keep an Activity (header + bullets) and a worked Example (+ Solution +
    equations) together."""
    out, in_example = [], False
    for p in paragraphs:
        is_example = bool(re.match(r"^Example\s\d+\.\d+", p))
        glue = (p.startswith("- ") or p.startswith("Solution")
                or (in_example and not is_example and "=" in p and len(p) <
400))
        if out and glue:
            out[-1] += "\n" + p
        else:
            out.append(p)
        in_example = is_example or (in_example and glue)
    return out

def clean_text(text: str) -> str:
    # 1. Running header at top of page: "Science82", "Science194 194194
194194", "Life Processes 95"
    text = re.sub(
        r"^s*(?:Science|Life Processes|Electricity)\s?\d{1,3}(?:[
\t]+\d{1,6})*[ \t]*\n",
        "", text
    )
    # 1b. Chapter-opening page: "Life Processes 5CHAPTER H ow do..." -> "How do.
.. "
    text = re.sub(r"^s*(?:Life Processes|Electricity)\s?\d{1,2}\s?
CHAPTER\s*([A-Z])\s?([a-z]+)", r"\1\2", text)

    # 2. Collapse repeated captions ("Figure 5.2Figure 5.2")
    text = re.sub(r"((?:Figure|Activity|Table)\s?\d+\.\d+)\1+", r"\1", text)

    # 3. Broken bullets and reprint footer
    text = text.replace("/square6", "- ")
    text = text.replace("...", "...")
    text = re.sub(r"Reprint\s+\d{4}-\d{2}", "", text)

    # 4. Rebuild paragraphs from broken lines
    caption_start = re.compile(r"^(?:Figure|Activity|Table)\s?\d+\.\d+")
    block_start = re.compile(r"^(?:\d{1,2}\.\s*[A-Z]|Example\s\d+\.
\d+|Solution\b)")
    heading = re.compile(r"^\d+\.\d+\s+[A-Z] [A-Z ,'\-\-]{4,}")

    def split_heading(line):
        toks = line.split(" ")

```

```

        i = next((k for k, t in enumerate(toks) if re.search(r"[a-z]", t)),
↳len(toks))
        return " ".join(toks[:i]), " ".join(toks[i:])

def dedupe_repeats(line: str) -> str:
    # overprinted / drop-cap headings: collapse repeats in the HEADING part
↳only, never in body text
    if re.match(r"^\d+\.\d+\s+[A-Z]{2,}", line):
        head, rest = split_heading(line)
        head = re.sub(r"(.{3,}?)\s?\1+", r"\1", head)
        head = re.sub(r"([A-Z])\s?\1{3,}", r"\1", head)
        line = (head + " " + rest).strip()
    line = re.sub(r"^(.{5,}?)\s?\1+$", r"\1", line)
    line = re.sub(r"^((?:\d+\.\d+\s+)?[A-Z][A-Z ,'\--?]{4,}?)\s?(?:\1\s?
↳)+", r"\1 ", line)
    return line.strip()

lines = [dedupe_repeats(ln.strip()) for ln in text.split("\n") if ln.
↳strip()]
lines = [ln for ln in lines if ln not in ("?", "QUESTIONS")] #
↳NEW: drop the page-icon "?" lines

paragraphs = []
for line in lines:
    if not paragraphs:
        paragraphs.append(line)
        continue
    prev = paragraphs[-1]
    starts_new_block = (
        line.startswith("- ")
        or prev.endswith((".", "?", "!", ":"))
        or caption_start.match(line)
        or block_start.match(line)
        or heading.match(line)
        or heading.match(prev)
    )
    if starts_new_block:
        paragraphs.append(line)
    else:
        paragraphs[-1] = prev + " " + line

# Keep Activities, worked Examples and bullet lists together
paragraphs = glue_blocks(paragraphs)

# Keep each heading together with the paragraph below it
merged = []
for p in paragraphs:

```

```

        if merged and heading.match(merged[-1]) and "\n" not in merged[-1]:
            merged[-1] += "\n" + p
        else:
            merged.append(p)

    return "\n\n".join(merged)

```

```

[13]: # Test it on one page: raw vs clean
raw = docs["doc2_electricity.pdf"][5].page_content
cleaned = clean_text(raw)
print("RAW (first 700 chars):\n", raw[:700])
print("\n" + "=" * 60 + "\n")
print("CLEAN (first 700 chars):\n", cleaned[:700])

```

RAW (first 700 chars):

Science176

Figure 11.3Figure 11.3Figure 11.3Figure 11.3Figure 11.3

V-I graph for a nichrome wire. A

straight line plot shows that as the current through a wire increases, the potential difference across the wire increases linearly - this is Ohm's law.

Activity 11.2Activity 11.2Activity 11.2Activity 11.2Activity 11.2

/square6Take a nichrome wire, a torch bulb, a 10 W bulb and an ammeter (0 - 5 A range), a plug

key and some connecting wires.

/square6Set up the circuit by connecting four dry cells of 1.5 V each in series with the ammeter

leaving a gap XY in the circuit, as shown in Fig. 11.4.

In this Activity, you will find that approximately the same value for V/I is obtained in each case. Thu

=====

CLEAN (first 700 chars):

Figure 11.3 V-I graph for a nichrome wire. A straight line plot shows that as the current through a wire increases, the potential difference across the wire increases linearly - this is Ohm's law.

Activity 11.2

- Take a nichrome wire, a torch bulb, a 10 W bulb and an ammeter (0 - 5 A range), a plug key and some connecting wires.

- Set up the circuit by connecting four dry cells of 1.5 V each in series with the ammeter leaving a gap XY in the circuit, as shown in Fig. 11.4.

In this Activity, you will find that approximately the same value for V/I is obtained in each case. Thus the V-I graph is a straight line that passes through the origin of the graph, as shown in Fig. 11.3. Thus, V/I is a

```
[14]: # Apply to every page (keep only the useful metadata)
docs_clean = {}
for name, pages in docs.items():
    docs_clean[name] = [
        Document(
            page_content=clean_text(p.page_content),
            metadata={
                "source": Path(p.metadata["source"]).name,
                "page": p.metadata["page"],
                "page_label": p.metadata.get("page_label"),
            },
        )
        for p in pages
    ]

for name, pages in docs_clean.items():
    text = "\n".join(p.page_content for p in pages)
    print(name)
    print(" '/square' left      :", text.count("/square"))
    print(" Repeated captions :", len(re.findall(r"((?:
↳Figure|Activity|Table)\s?\d+\.\d+)\1{1,})", text)))
    print(" Double newlines      :", text.count("\n\n"))
    print(" Single newlines      :", text.count("\n") - 2 * text.count("\n\n"))
```

```
doc1_life_processes.pdf
'/square' left      : 0
Repeated captions   : 0
Double newlines     : 176
Single newlines     : 88
doc2_electricity.pdf
'/square' left      : 0
Repeated captions   : 0
Double newlines     : 132
Single newlines     : 104
```

1.10 Section 10: Verify the Cleaning

Expected: no page starts with a header, 0 repeated headings, 0 /square. The paragraph audit shows a few lowercase starts. These are sentences that continue on the next page. The page-joining step in Section 11 fixes them, so the real check is on chunks.

```
[15]: # Header, heading and bullet checks
hdr = re.compile(r"(Science|Life Processes|Electricity)\s?\d{1,3}\b")
heading_dup = re.compile(r"^(\\d+\\.\\d+\\s+[A-Z] [A-Z , '\\-]{1,}?)\\s?\\1", flags=re.
↳M)

for name, pages in docs_clean.items():
    text = "\n".join(p.page_content for p in pages)
```

```

    print(name)
    print("  pages still starting with a header:", [p.metadata["page"] for p in
↪pages if hdr.match(p.page_content)])
    print("  repeated headings                :", len(heading_dup.
↪findall(text)))
    print("  '/square' left                    :", text.count("/square"))

# Spot check a page that had a messy header ("Science194 194194 194194")
print()
print(repr(docs["doc2_electricity.pdf"][23].page_content[:150]))
print(repr(docs_clean["doc2_electricity.pdf"][23].page_content[:150]))

```

```

doc1_life_processes.pdf
  pages still starting with a header: []
  repeated headings                : 0
  '/square' left                    : 0
doc2_electricity.pdf
  pages still starting with a header: []
  repeated headings                : 0
  '/square' left                    : 0

```

'Science194 194194 194194\n15. Two lamps, one rated 100 W at 220 V, and the other 60 W at 220 V, are connected\nin parallel to electric mains supply. Wha'

'15. Two lamps, one rated 100 W at 220 V, and the other 60 W at 220 V, are connected in parallel to electric mains supply. What current is drawn from t'

```

[16]: # Paragraph audit
def audit(pages_dict):
    for name, pages in pages_dict.items():
        paras = [p for pg in pages for p in pg.page_content.split("\n\n")]
        lower_start = [p[:70] for p in paras if p[:1].islower()]
        abbr_end = [p[-50:] for p in paras if re.search(r"\b(Fig|Figs|e\.g|i\.\
↪e|etc)\. $", p)]
        print(f"\n{name}: {len(paras)} paragraphs")
        print("  Start with lowercase (continue from previous page):",
↪len(lower_start))
        for s in lower_start[:5]:
            print("    ", repr(s))
        print("  End with an abbreviation:", len(abbr_end))
        for s in abbr_end[:5]:
            print("    ", repr(s))

audit(docs_clean)

```

```

doc1_life_processes.pdf: 197 paragraphs
  Start with lowercase (continue from previous page): 10
    'class, even if we are just asleep, this maintenance job has to go on.'

```

```

'design in multi-cellular organisms are followed, a specialised tissue '
'taken up in the form of inorganic nitrates or nitrites. Or it is taken'
'have often heard adults complaining about 'acidity'. Can this be relat'
'process breaks up the three-carbon pyruvate molecule to give three mol'
End with an abbreviation: 0

```

doc2_electricity.pdf: 156 paragraphs

```

Start with lowercase (continue from previous page): 1
'have very low resistivity in the range of 10-8 Ω m to 10-6 Ω m. They'
End with an abbreviation: 1
'evices such as electric heater, electric iron etc.'

```

1.11 Section 11: Chunking Helpers

- **Sentence-aware separators** so chunks end at sentence boundaries.
- **keep_separator="end"** so the full stop stays at the end of its own sentence (the default puts it at the start of the next chunk).
- **Pages are joined before splitting**, so a sentence that crosses a page break stays whole. Each chunk is mapped back to its page.
- **merge_tiny** attaches tiny chunks (headings, tail fragments) to a neighbour.

```

[17]: # Splitter, page joining, section-aware chunking and tiny-chunk merge
SEPARATORS = ["\n\n", "\n", ". ", "? ", "; ", " ", ""]
SECTION_TITLES = {
    "doc1_life_processes.pdf": {
        "5.1": "5.1 WHAT ARE LIFE PROCESSES?", "5.2": "5.2 NUTRITION", "5.3": "5.3 RESPIRATION",
        "5.4": "5.4 TRANSPORTATION", "5.5": "5.5 EXCRETION"},
    "doc2_electricity.pdf": {
        "11.1": "11.1 ELECTRIC CURRENT AND CIRCUIT", "11.2": "11.2 ELECTRIC POTENTIAL AND POTENTIAL DIFFERENCE",
        "11.3": "11.3 CIRCUIT DIAGRAM", "11.4": "11.4 OHM'S LAW",
        "11.5": "11.5 FACTORS ON WHICH THE RESISTANCE OF A CONDUCTOR DEPENDS",
        "11.6": "11.6 RESISTANCE OF A SYSTEM OF RESISTORS", "11.7": "11.7 HEATING EFFECT OF ELECTRIC CURRENT",
        "11.8": "11.8 ELECTRIC POWER"},
}

def fix_title(src, sec):
    if sec:
        m = re.match(r"^(?=\d+\.\d+)", sec)
        if m and m.group(1) in SECTION_TITLES.get(src, {}):
            return SECTION_TITLES[src][m.group(1)]
    return sec

def clean_heading(line):
    """'11.4 OHM'S LA W WW Is there a relat...' -> '11.4 OHM'S LA W WW'"""
    m = re.match(r"^(?=\d+\.\d+)\s+(?:\b[A-Z][A-Z']\s*|(?=\s|$)\s*)+", line)

```

```

    return f"{m.group(1)} {m.group(2).strip()}" if m else line.strip()[:60]

EXTRA = r"(?:WHAT YOU HAVE LEARNT|What you have learnt|EXERCISES)"
HEADING_SPLIT = re.compile(r"(?m)^(?=\d+\. \d+\s+[A-Z]{2,}|" + EXTRA + r"\s*$)")
    ↪      # CHANGED: 2+ capitals, so "5.4 TR ..." is caught
HEAD_LINE     = re.compile(r"\d+\. \d+\s+[A-Z][^\n]*|" + EXTRA)
PAGE_STARTS_WITH_HEADING = re.compile(r"(?m)(?:\d+\. \d+\s+[A-Z]{2,}|" + EXTRA +
    ↪r"\s*$)") # NEW

def merge_tiny(chunks, min_len=100):
    """Tiny chunk starting lowercase = tail of previous chunk -> merge back.
    Tiny chunk starting uppercase (heading / caption / question) -> merge
    ↪with the next chunk."""
    out, carry = [], None
    for c in chunks:
        if carry is not None:
            c.page_content = carry.page_content + "\n" + c.page_content
            c.metadata["start_index"] = carry.metadata["start_index"]
            carry = None
        if len(c.page_content) < min_len:
            if c.page_content[:1].islower() and out:
                out[-1].page_content += " " + c.page_content
            else:
                carry = c
                continue
        out.append(c)
    if carry is not None:
        if out:
            out[-1].page_content += "\n" + carry.page_content
        else:
            out.append(carry)
    return out

def make_splitter(size, overlap):
    return RecursiveCharacterTextSplitter(
        chunk_size=size,
        chunk_overlap=overlap,
        separators=SEPARATORS,
        keep_separator="end",
        add_start_index=True,
    )

def join_pages(pages):
    full, spans = "", []
    ↪# spans = (start_char,
    ↪page_number)
    for p in pages:
        if full:

```

```

        starts_lower = p.page_content[:1].islower()
        starts_heading = bool(PAGE_STARTS_WITH_HEADING.match(p.
↪page_content)) # NEW
        ends_clean = full.rstrip().endswith((".", "?", "!", ":"))
        join_with_space = (starts_lower or not ends_clean) and not
↪starts_heading # CHANGED
        full = full.rstrip() + (" " if join_with_space else "\n\n")
        spans.append((len(full), p.metadata["page"]))
        full += p.page_content
        return full, spans

def chunk_document(pages, splitter, by_section=True):
    """Join pages, split per section (chunks never cross a heading), merge tiny
↪chunks, map back to pages."""
    full, spans = join_pages(pages)
    bounds = sorted(set([0, len(full)] + ([m.start() for m in HEADING_SPLIT.
↪finditer(full)] if by_section else [])))
    chunks = []
    for a, b in zip(bounds, bounds[1:]):
        sec_text = full[a:b]
        if not sec_text.strip():
            continue
        m = HEAD_LINE.match(sec_text)
        part = splitter.create_documents(
            [sec_text],
            metadatas=[{"source": pages[0].metadata["source"],
                        "section": fix_title(pages[0].metadata["source"],
↪clean_heading(m.group(0))) if m else None,}],
        )
        for c in part:
            c.metadata["start_index"] += a
            chunks.extend(merge_tiny(part))

    # runs once, after ALL sections are done (note: 4 spaces, same level as
↪"for a, b")
    for c in chunks:
        s = c.metadata["start_index"]
        c.metadata["page"] = max(pg for st, pg in spans if st <= s)
        c.page_content = c.page_content.lstrip(" .,:;?!\\n") # never start
↪with punctuation
        if not c.metadata.get("section"):
            c.metadata["section"] = "Chapter opening" # text before
↪the first numbered heading
    return chunks

```

```
[18]: # Measuring helpers
def chunk_stats(chunks):
    lengths = [len(c.page_content) for c in chunks]
    no_end = sum(1 for c in chunks if c.page_content.strip()[-1:] not in ".?!:;")
    return {
        "chunks": len(chunks), "min": min(lengths), "max": max(lengths),
        "avg": round(sum(lengths) / len(lengths)),
        "no_end_punct_%": round(100 * no_end / len(chunks)),
        "tiny_<100": sum(1 < 100 for l in lengths),
    }

def get_paragraphs(name, min_len=80):
    return [p for pg in docs_clean[name] for p in pg.page_content.split("\n\n")
    if len(p) > min_len]

def paragraph_integrity(chunks, name):
    """% of real paragraphs that sit completely inside one chunk."""
    paras = get_paragraphs(name)
    kept = sum(1 for p in paras
        if any(p[:40] in c.page_content and p[-40:] in c.page_content
    for c in chunks))
    return round(100 * kept / len(paras))

def find_chunks(chunks, phrase):
    """Print every chunk that contains the phrase."""
    for i, c in enumerate(chunks):
        if phrase.lower() in c.page_content.lower():
            print(f"--- chunk {i} | page {c.metadata['page']} | {len(c.
    page_content)} chars ---")
            print(c.page_content, "\n")
```

1.12 Section 12: Check the Chunks

```
[19]: # Chunk quality check (size 1000, overlap 100)
all_ok = True
for name, pages in docs_clean.items():
    ch = chunk_document(pages, make_splitter(1000, 100))
    tiny = sum(len(c.page_content) < 100 for c in ch)
    lower = sum(c.page_content[:1].islower() for c in ch)
    punct = sum(c.page_content[:1] in ".?;,:;" for c in ch)
    no_end = sum(c.page_content.strip()[-1:] not in ".?!:;)" for c in ch)
    print(name, "-", len(ch), "chunks")
    print(" tiny chunks (<100)           :", tiny, "(want 0)")
    print(" start lowercase                 :", lower, "(want 0)")
    print(" start with punctuation             :", punct, "(want 0)")
```

```

    print(" not ending with . ? ! : ; ) :", no_end, "(known issue, see Section 19)")
    all_ok = all_ok and tiny == 0 and lower == 0 and punct == 0
    print("CHECK:", "PASS" if all_ok else "FAIL")

```

```

doc1_life_processes.pdf - 70 chunks
  tiny chunks (<100)          : 0 (want 0)
  start lowercase             : 0 (want 0)
  start with punctuation      : 0 (want 0)
  not ending with . ? ! : ; ) : 6 (known issue, see Section 19)
doc2_electricity.pdf - 61 chunks
  tiny chunks (<100)          : 0 (want 0)
  start lowercase             : 0 (want 0)
  start with punctuation      : 0 (want 0)
  not ending with . ? ! : ; ) : 3 (known issue, see Section 19)
CHECK: PASS
- 70 chunks
  tiny chunks (<100)          : 0 (want 0)
  start lowercase             : 0 (want 0)
  start with punctuation      : 0 (want 0)
  not ending with . ? ! : ; ) : 6 (known issue, see Section 19)
doc2_electricity.pdf - 61 chunks
  tiny chunks (<100)          : 0 (want 0)
  start lowercase             : 0 (want 0)
  start with punctuation      : 0 (want 0)
  not ending with . ? ! : ; ) : 3 (known issue, see Section 19)
CHECK: PASS

```

```

[20]: # Chunks that still look odd (size 1000, overlap 100), with what comes next
for name, pages in docs_clean.items():
    ch = chunk_document(pages, make_splitter(1000, 100))
    for i, c in enumerate(ch):
        t = c.page_content.strip()
        problems = []
        if t[:1] in ".?;,:": problems.append("starts with punctuation")
        if t[:1].islower(): problems.append("starts lowercase")
        if t[-1:] not in ".?!:;)" : problems.append("no end punctuation")
        if problems:
            nxt = ch[i + 1].page_content[:80] if i + 1 < len(ch) else "(last_
            chunk)"
            print(f"{name[:12]} | chunk {i} | page {c.metadata['page']} | {'', ' '.
            join(problems)}")
            print(" ENDS : ", repr(t[-90:]))
            print(" NEXT STARTS: ", repr(nxt))

```

```

doc1_life_pr | chunk 10 | page 2 | no end punctuation
  ENDS : '2 Variegated leaf (a) before and (b) after starch test\n\nFigure
5.1 Cross-section of a leaf'

```

NEXT STARTS: 'Figure 5.1 Cross-section of a leaf\n\nActivity 5.1\n- Take a potted plant with vari'

doc1_life_pr | chunk 21 | page 6 | no end punctuation

ENDS : 'the inner lining of the stomach from the action of the acid under normal conditions. We'

NEXT STARTS: 'Figure 5.6 Human alimentary canal have often heard adults complaining about 'ac'

doc1_life_pr | chunk 26 | page 8 | no end punctuation

ENDS : 'digested food?\n\nTube Test tube containing lime waterLime water Rubber tube (a) (b) Pichkari'

NEXT STARTS: 'Tube Test tube containing lime waterLime water Rubber tube (a) (b) Pichkari\n\nAct'

doc1_life_pr | chunk 28 | page 8 | no end punctuation

ENDS : 'anaerobic respiration. Break-down of pyruvate using oxygen takes place in the mitochondria. This'

NEXT STARTS: 'Figure 5.7 (a) Air being passed into lime water with a pichkari/ syringe, (b) air'

doc1_life_pr | chunk 55 | page 16 | no end punctuation

ENDS : 'This is explained by simple physical forces, the translocation in phloem is achieved by utilising'

NEXT STARTS: 'Figure 5.12 Movement of water during transpiration in a tree energy. Material list'

doc1_life_pr | chunk 58 | page 17 | no end punctuation

ENDS : 'Blood in the kidneys. It is then no surprise that the basic filtration unit in the kidneys,'

NEXT STARTS: 'Figure 5.13 Excretory system in human beings like in the lungs, is a cluster of '

doc2_electri | chunk 19 | page 7 | no end punctuation

ENDS : 'Resistivity is Ω m. It is a characteristic property of the material. The metals and alloys'

NEXT STARTS: 'Figure 11.5 Figure 11.5 Figure 11.5 Figure 11.5 Figure 11.5 Electric circuit to '

doc2_electri | chunk 40 | page 16 | no end punctuation

ENDS : '\nFigure 11.12 An electric circuit showing the combination of series and parallel resistors'

NEXT STARTS: '11.7 HEATING EFFECT OF ELECTRIC CURRENT\nWe know that a battery or a cell is a source'

doc2_electri | chunk 56 | page 21 | no end punctuation

ENDS : 'The energy to operate the refrigerator for 30 days is $96 \text{ kWh} \times \text{Rs } 3.00 \text{ per kWh} = \text{Rs } 288.00$ '

NEXT STARTS: 'EXERCISES\n\n1. A piece of wire of resistance R is cut into five equal parts. The'

ENDS : '2 Variegated leaf (a) before and (b) after starch test\n\nFigure 5.1 Cross-section of a leaf'

NEXT STARTS: 'Figure 5.1 Cross-section of a leaf\n\nActivity 5.1\n- Take a potted plant with vari'

doc1_life_pr | chunk 21 | page 6 | no end punctuation

ENDS : 'the inner lining of the stomach fr om the action of the acid under nor mal conditions. W e'

NEXT STARTS: 'Figure 5.6 Human alimentary canal have often heard adults complaining about 'ac'

doc1_life_pr | chunk 26 | page 8 | no end punctuation

ENDS : 'igested food?\n\nTube Test tube containing lime waterLime water Rubber tube (a) (b) Pichkari'

NEXT STARTS: 'Tube Test tube containing lime waterLime water Rubber tube (a) (b) Pichkari\n\nAct'

doc1_life_pr | chunk 28 | page 8 | no end punctuation

ENDS : 'ic respiration. Break- down of pyruvate using oxygen takes place in the mitochondria. This'

NEXT STARTS: 'Figure 5.7 (a) Air being passed into lime water with a pichkari/ syringe, (b) ai'

doc1_life_pr | chunk 55 | page 16 | no end punctuation

ENDS : ' explained by simple physical forces, the translocation in phloem is achieved by utilising'

NEXT STARTS: 'Figure 5.12 Movement of water during transpiration in a tree energy. Material li'

doc1_life_pr | chunk 58 | page 17 | no end punctuation

ENDS : 'lood in the kidneys. It is then no surprise that the basic filtration unit in the kidneys,'

NEXT STARTS: 'Figure 5.13 Excretory system in human beings like in the lungs, is a cluster of '

doc2_electri | chunk 19 | page 7 | no end punctuation

ENDS : 'resistivity is Ω m. It is a characteristic property of the material. The metals and alloys'

NEXT STARTS: 'Figure 11.5 Figure 11.5 Figure 11.5 Figure 11.5 Figure 11.5 Electric circuit to '

doc2_electri | chunk 40 | page 16 | no end punctuation

ENDS : '\nFigure 11.12 An electric circuit showing the combination of series and parallel resistors'

NEXT STARTS: '11.7 HEA TING EFFECT OF ELECTRIC CURRENT\nWe know that a battery or a cell is a s'

doc2_electri | chunk 56 | page 21 | no end punctuation

ENDS : 'f energy to operate the refrigerator for 30 days is $96 \text{ kW h} \times \text{Rs } 3.00 \text{ per kW h} = \text{Rs } 288.00$ '

NEXT STARTS: 'EXERCISES\n\n1. A piece of wire of resistance R is cut into five equal parts. The'

```
[21]: # Raw vs clean vs clean + new chunking (same size 500, so the comparison is
      ↪fair)
splitter_500 = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks_raw = {n: splitter_500.split_documents(p) for n, p in docs.items()}
chunks_clean_old = {n: splitter_500.split_documents(p) for n, p in docs_clean.
      ↪items() }
```

```

chunks_clean = {n: chunk_document(p, make_splitter(500, 50)) for n, p in docs_clean.items()}

cleaning_rows = []
for name in docs:
    for label, ch in [("raw", chunks_raw),
                      ("clean + old splitter", chunks_clean_old),
                      ("clean + new chunking", chunks_clean)]:
        cleaning_rows.append({"doc": name[:12], "version": label,
                              **chunk_stats(ch[name])})

cleaning_table = pd.DataFrame(cleaning_rows)
cleaning_table

```

```

[21]:
      doc          version  chunks  min  max  avg  no_end_punct_% \
0  doc1_life_pr         raw     124   15  499  434             79
1  doc1_life_pr  clean + old splitter    144   14  499  359             24
2  doc1_life_pr  clean + new chunking    140  108  537  363              5
3  doc2_electri         raw     109   50  499  428             72
4  doc2_electri  clean + old splitter    125   33  499  350             30
5  doc2_electri  clean + new chunking    117  104  546  366              6

tiny_<100
0      2
1      5
2      0
3      2
4      9
5      0

```

```

[22]: # Look at real chunks: where does "One volt" end up at size 500?
      find_chunks(chunks_clean["doc2_electricity.pdf"], "One volt")

```

--- chunk 14 | page 2 | 175 chars ---

One volt is the potential difference between two points in a current carrying conductor when 1 joule of work is done to move a charge of 1 coulomb from one point to the other.

1.13 Section 13: Gold Questions

A small test set. Each question lists phrases that must appear **together in one chunk**. A chunk size that splits the evidence fails the question. Phrases must be copied exactly from the cleaned text and must sit inside one sentence or one line.

```

[23]: # Write the gold questions
      # type: definition / formula / fact / list / process / reasoning / activity /
      # example

```

```

# alt: other phrase sets that also answer the question (used by Experiment 2,
↳not by the chunking score)
gold = [
    # ----- Electricity -----
    {"doc": "doc2_electricity.pdf", "type": "definition", "question": "What is
↳one volt?",
    "must_contain": ["One volt", "1 coulomb"]},
    {"doc": "doc2_electricity.pdf", "type": "formula", "question": "What is the
↳formula for potential difference?",
    "must_contain": ["Potential difference (V) between two points", "V = W/
↳Q"]},
    {"doc": "doc2_electricity.pdf", "type": "fact", "question": "Which
↳instrument measures potential difference and how is it connected?",
    "must_contain": ["voltmeter", "connected in parallel"]},
    {"doc": "doc2_electricity.pdf", "type": "definition", "question": "State
↳Ohm's law.",
    "must_contain": ["directly proportional to the current", "temperature
↳remains the same"]},
    {"doc": "doc2_electricity.pdf", "type": "fact", "question": "After whom is
↳the volt named?",
    "must_contain": ["Alessandro Volta", "volt"]},
    {"doc": "doc2_electricity.pdf", "type": "example", "question": "Example 11.
↳1: charge flowing for 0.5 A in 10 minutes?",
    "must_contain": ["0.5 A is drawn", "300 C"]},
    {"doc": "doc2_electricity.pdf", "type": "example", "question": "Example 11.
↳1: what are the given values?",
    "must_contain": ["Find the amount of electric charge", "I = 0.5 A; t = 10
↳min = 600 s"]},
    {"doc": "doc2_electricity.pdf", "type": "formula", "question": "What is the
↳relation between V, I and R?",
    "must_contain": ["V I", "V = IR"]},
    {"doc": "doc2_electricity.pdf", "type": "definition", "question": "What is
↳resistance?",
    "must_contain": ["is called its resistance", "property of a conductor to
↳resist the flow of charges"],
    "alt": [{"Resistance is a property that resists the flow of electrons"}]},
    {"doc": "doc2_electricity.pdf", "type": "fact", "question": "What does the
↳V-I graph of a metallic wire look like?",
    "must_contain": ["V-I graph is a straight line", "passes through the
↳origin"]},
    {"doc": "doc2_electricity.pdf", "type": "activity", "question": "Activity
↳11.2: what apparatus and set-up are used?",
    "must_contain": ["Take a nichrome wire, a torch bulb", "Set up the circuit
↳by connecting four dry cells"]},
    {"doc": "doc2_electricity.pdf", "type": "fact", "question": "Who found the
↳relationship between current and potential difference?",

```

```

    "must_contain": ["Georg Simon Ohm", "found out the relationship between
↳the current"]},
    {"doc": "doc2_electricity.pdf", "type": "fact", "question": "How many
↳joules is 1 kW h?"},
    "must_contain": ["1 kW h", "3.6 × 106 J"]},
    {"doc": "doc2_electricity.pdf", "type": "formula", "question": "What is the
↳formula for electrical energy dissipated in a resistor?"},
    "must_contain": ["The electrical energy dissipated in a resistor is given
↳by  $W = V \times I \times t$ ", "The unit of power is watt"]},

    # ----- Life processes -----
    {"doc": "doc1_life_processes.pdf", "type": "list", "question": "What are
↳the events of photosynthesis?"},
    "must_contain": ["Absorption of light energy", "Reduction of carbon
↳dioxide"]},
    {"doc": "doc1_life_processes.pdf", "type": "activity", "question": "How do
↳you test for starch in a leaf (Activity 5.1)?"},
    "must_contain": ["Take a potted plant with variegated leaves", "dilute
↳solution of iodine"]},
    {"doc": "doc1_life_processes.pdf", "type": "definition", "question": "What
↳is respiration?"},
    "must_contain": ["break -down of food sources", "what we call
↳respiration"],
    "alt": [["organic compounds such as glucose are broken down to provide
↳energy"]]},
    {"doc": "doc1_life_processes.pdf", "type": "process", "question": "How does
↳root pressure move water upward?"},
    "must_contain": ["steady movement of water into root xylem", "column of
↳water"]},
    {"doc": "doc1_life_processes.pdf", "type": "process", "question": "What
↳happens in the second event of photosynthesis?"},
    "must_contain": ["Conversion of light energy to chemical energy",
↳"splitting of water molecules into hydrogen and oxygen"]},
    {"doc": "doc1_life_processes.pdf", "type": "reasoning", "question": "Why is
↳the plant kept in a dark room for three days?"},
    "must_contain": ["dark room for three days", "starch gets used up"]},
    {"doc": "doc1_life_processes.pdf", "type": "activity", "question": "What is
↳done to the leaf before the iodine test?", # evidence spans 3 bullets
    "must_contain": ["Dip the leaf in boiling water", "immerse it in a beaker
↳containing alcohol", "dilute solution of iodine"]},
    {"doc": "doc1_life_processes.pdf", "type": "reasoning", "question": "Is
↳root pressure enough to move water to great heights?"},
    "must_contain": ["this pressure by itself is unlikely to be enough", "move
↳water in the xylem upwards"]},

```

```

    {"doc": "doc1_life_processes.pdf", "type": "activity", "question": "
    ↪Activity 5.8: how is moisture loss from the soil prevented?",
    "must_contain": ["Take two small pots", "plastic sheet so that moisture
    ↪cannot escape by evaporation"]},
    {"doc": "doc1_life_processes.pdf", "type": "definition", "question": "What
    ↪are life processes?",
    "must_contain": ["perform this maintenance job are life processes",
    ↪energy is needed for them"]},
    {"doc": "doc1_life_processes.pdf", "type": "definition", "question": "What
    ↪is translocation and where does it occur?",
    "must_contain": ["is called translocation", "phloem"]},
    {"doc": "doc1_life_processes.pdf", "type": "fact", "question": "What else
    ↪does the phloem transport besides products of photosynthesis?",
    "must_contain": ["the phloem transports amino acids and other
    ↪substances"]},
    {"doc": "doc1_life_processes.pdf", "type": "activity", "question": "What is
    ↪the purpose of potassium hydroxide in the CO2 activity?",
    "must_contain": ["Take two healthy potted plants", "potassium hydroxide is
    ↪used to absorb carbon dioxide"]},
    {"doc": "doc1_life_processes.pdf", "type": "activity", "question": "How is
    ↪the set-up made air-tight in the CO2 activity?",
    "must_contain": ["Use vaseline to seal the bottom of the jars", "Cover
    ↪both plants with separate bell-jars"]},
]

questions = [g["question"] for g in gold]
assert len(questions) == len(set(questions)), "duplicate question in gold list"
ALLOWED_TYPES = {"definition", "formula", "fact", "list", "process",
    ↪"reasoning", "activity", "example"}
assert all(g.get("type") in ALLOWED_TYPES for g in gold), "every question needs
    ↪a valid type"
print("Gold questions written in code:", len(gold))
print(pd.Series([g["type"] for g in gold]).value_counts().to_string())

```

Gold questions written in code: 28

definition	6
fact	6
activity	6
formula	3
example	2
process	2
reasoning	2
list	1

```

[24]: # Add the 14 questions that were the first held-out set (they are now
    ↪development questions)

```

```

used_path = EVAL_DIR / "gold_questions_heldout_v1_USED.json"
if used_path.exists():
    with open(used_path, encoding="utf-8") as f:
        used = json.load(f)
        have = {g["question"] for g in gold}
        added = [{**g, "origin": "heldout_v1"} for g in used if g["question"] not_
↳in have]
        gold = gold + added
        print("Added", len(added), "| gold questions now:", len(gold))
assert all(g.get("type") in ALLOWED_TYPES for g in gold), "every question needs_
↳a valid type"
assert len({g["question"] for g in gold}) == len(gold), "duplicate question"

```

Added 14 | gold questions now: 42
14 | gold questions now: 42

```

[25]: # Validate the phrases, save the valid gold set, define gold_ok / gold_score /_
↳gold_failures
full_text = {n: " ".join(p.page_content for p in pages).lower() for n, pages in_
↳docs_clean.items()}

valid_gold, dropped, alt_dropped = [], [], []
for g in gold:
    text = full_text[g["doc"]]
    missing = [p for p in g["must_contain"] if p.lower() not in text]
    if missing:
        dropped.append((g["question"], missing))          # main evidence_
↳missing -> drop the question
        continue
    g = dict(g)
    if "alt" in g:                                         # alternative_
↳evidence: keep only the sets that exist
        ok_sets = [s for s in g["alt"] if all(p.lower() in text for p in s)]
        if len(ok_sets) < len(g["alt"]):
            alt_dropped.append(g["question"])
        g["alt"] = ok_sets
    valid_gold.append(g)

for q, m in dropped:
    print("DROPPED (phrase not in cleaned text):", q, "->", m)
for q in alt_dropped:
    print("WARNING: an alt evidence set was removed (phrase not in cleaned_
↳text):", q)
print("CHECK:", "PASS" if not dropped and not alt_dropped else f"{len(dropped)}_
↳question(s) dropped, {len(alt_dropped)} alt set(s) removed - fix the phrases_
↳using find_chunks / the cleaned text")

```

```

gold = valid_gold
EVAL_DIR.mkdir(parents=True, exist_ok=True)
gold_path = EVAL_DIR / "gold_questions.json"
with open(gold_path, "w", encoding="utf-8") as f:
    json.dump(gold, f, indent=2, ensure_ascii=False)
print("Saved", len(gold), "questions to", gold_path)

# Chunking score is STRICT on purpose: it only checks that the main evidence
↳ sits inside one chunk.
# "alt" is used later, in Experiment 2, when judging whether a retrieved chunk
↳ answers the question.
def gold_ok(g, chunks):
    return any(all(p.lower() in c.page_content.lower() for p in
↳ g["must_contain"]) for c in chunks)

def gold_score(chunks_by_doc):
    ok = sum(gold_ok(g, chunks_by_doc[g["doc"]]) for g in gold)
    return round(100 * ok / len(gold))

def gold_failures(chunks_by_doc):
    failed = [g["question"] for g in gold if not gold_ok(g,
↳ chunks_by_doc[g["doc"]])]
    if failed:
        for q in failed:
            print(" FAIL:", q)
    else:
        print(" all gold facts kept together")

```

CHECK: PASS

Saved 42 questions to

c:\Users\ASUS\Desktop\VisualMind\data\eval\gold_questions.json

1.14 Section 14: Paragraph Lengths

How long are the real paragraphs? This tells us what chunk size is reasonable.

```

[26]: # Paragraph length statistics
for name in docs_clean:
    lens = [len(p) for p in get_paragraphs(name)]
    print(name, "| paragraphs:", len(lens),
          "| median:", int(np.median(lens)),
          "| 75th pct:", int(np.percentile(lens, 75)),
          "| 90th pct:", int(np.percentile(lens, 90)),
          "| max:", max(lens))

```

doc1_life_processes.pdf | paragraphs: 143 | median: 281 | 75th pct: 434 | 90th
pct: 630 | max: 1488

doc2_electricity.pdf | paragraphs: 131 | median: 201 | 75th pct: 407 | 90th pct:

1.15 Section 15: Chunk Size Experiment

Overlap is 12% of the size here, so the sizes are compared fairly. Overlap is tested separately in Section 16. Columns: gold_% = gold facts kept in one chunk, paras_whole_% = paragraphs kept whole. paras_whole_% rises with size, but gold_% can drop again at the largest size, so also watch avg_len and the token columns. Very large chunks make retrieval less precise and can exceed an embedding model's token limit.

```
[27]: # Size grid
sizes = [500, 800, 1000, 1200]
chunks_by_size, rows = {}, []

for size in sizes:
    overlap = round(size * 0.12)
    by_doc = {n: chunk_document(p, make_splitter(size, overlap)) for n, p in docs_clean.items()}
    for n, ch in by_doc.items():
        chunks_by_size[(n, size)] = ch
    allc = [c for ch in by_doc.values() for c in ch]
    lens = [len(c.page_content) for c in allc]
    rows.append({
        "size": size, "overlap": overlap, "chunks": len(allc),
        "avg_len": round(np.mean(lens)), "max_len": max(lens),
        "tiny_<100": sum(l < 100 for l in lens),
        "gold_%": gold_score(by_doc),
        "paras_whole_%": round(np.mean([paragraph_integrity(by_doc[n], n) for n in docs_clean])),
    })

size_table = pd.DataFrame(rows)
size_table
```

```
[27]:
```

	size	overlap	chunks	avg_len	max_len	tiny_<100	gold_%	paras_whole_%
0	500	60	257	365	546	0	93	80
1	800	96	164	580	892	0	100	93
2	1000	120	131	726	1066	0	100	96
3	1200	144	105	914	1194	0	98	98

```
[28]: enc = tiktoken.get_encoding("cl100k_base") # OpenAI tokenizer: an estimate
only, Experiment 2 checks the real model limit

def token_stats(size):
    ch = [c for n in docs_clean for c in chunks_by_size[(n, size)]]
    toks = [len(enc.encode(c.page_content)) for c in ch]
    return pd.Series({"avg_tokens": round(np.mean(toks)), "max_tokens": max(toks)})
```

```
size_table = size_table.drop(columns=["avg_tokens", "max_tokens"],
    errors="ignore")
size_table = size_table.join(size_table["size"].apply(token_stats))
size_table
```

```
[28]:
```

	size	overlap	chunks	avg_len	max_len	tiny_<100	gold_%	paras_whole_%	\
0	500	60	257	365	546	0	93	80	
1	800	96	164	580	892	0	100	93	
2	1000	120	131	726	1066	0	100	96	
3	1200	144	105	914	1194	0	98	98	

	avg_tokens	max_tokens
0	85	213
1	135	272
2	168	340
3	211	364

```
[29]: # Which gold questions fail at each size?
for size in sizes:
    print(f"size {size}")
    gold_failures({n: chunks_by_size[(n, size)] for n in docs_clean})
```

```
size 500
    FAIL: How do you test for starch in a leaf (Activity 5.1)?
    FAIL: What is done to the leaf before the iodine test?
    FAIL: What are life processes?
size 800
    all gold facts kept together
size 1000
    all gold facts kept together
size 1200
    FAIL: What are the events of photosynthesis?
```

```
[30]: # Eyeball 3 random chunks at size 1000
random.seed(42)
for c in random.sample(chunks_by_size[("doc1_life_processes.pdf", 1000)], 3):
    print("=" * 70)
    print(f"page {c.metadata['page']} | {len(c.page_content)} chars")
    print(c.page_content)
```

```
=====
page 4 | 723 chars
- Take two healthy potted plants which are nearly the same size.
- Keep them in a dark room for three days.
- Now place each plant on separate glass plates. Place a watch-glass containing
potassium hydroxide by the side of one of the plants. The potassium hydroxide is
used to absorb carbon dioxide.
```

- Cover both plants with separate bell-jars as shown in Fig. 5.4.
- Use vaseline to seal the bottom of the jars to the glass plates so that the set-up is air -tight.
- Keep the plants in sunlight for about two hours.
- Pluck a leaf from each plant and check for the presence of starch as in the above activity.
- Do both the leaves show the presence of the same amount of starch?
- What can you conclude from this activity?

=====

page 0 | 874 chars

5.1 WHAT ARE LIFE PROCESSES?

The maintenance functions of living organisms must go on even when they are not doing anything particular. Even when we are just sitting in class, even if we are just asleep, this maintenance job has to go on.

The processes which together perform this maintenance job are life processes.

Since these maintenance processes are needed to prevent damage and break-down, energy is needed for them. This energy comes from outside the body of the individual organism. So there must be a process to transfer a source of energy from outside the body of the organism, which we call food, to the inside, a process we commonly call nutrition.

If the body size of the organisms is to grow, additional raw material will also be needed from outside. Since life on earth depends on carbon- based molecules, most of these food sources are also carbon-based.

=====

page 10 | 887 chars

In human beings (Fig. 5.9), air is taken into the body through the nostrils. The air passing through the nostrils is filtered by fine hairs that line the passage. The passage is also lined with mucus which helps in this process. From here, the air passes through the throat and into the lungs. Rings of cartilage are present in the throat. These ensure that the air-passage does not collapse.

Using tobacco directly or any product of tobacco in the form of cigar, cigarettes, bidis, hookah, gutkha, etc., is harmful. Use of tobacco most commonly affects the tongue, lungs, heart and liver. Smokeless tobacco is also a major risk factor for heart attacks, strokes, pulmonary diseases and several forms of cancers.

There is a high incidence of oral cancer in India due to the chewing of tobacco in the form of gutkha. Stay healthy; just say NO to tobacco and its products!

More to Know!

```
[31]: # Which gold questions fail at 10% overlap, and do they pass at 15%?
      for size in [800, 1000, 1200]:
```

```

    for pct in [0.10, 0.15]:
        ov = round(size * pct)
        print(f"size {size}, overlap {ov} ({round(pct * 100)}%)")
        gold_failures({n: chunk_document(p, make_splitter(size, ov)) for n, p
↪in docs_clean.items()})

```

```

size 800, overlap 80 (10%)
    all gold facts kept together
size 800, overlap 120 (15%)
    all gold facts kept together
size 1000, overlap 100 (10%)
    FAIL: What is the purpose of potassium hydroxide in the CO2 activity?
size 1000, overlap 150 (15%)
    all gold facts kept together
size 1200, overlap 120 (10%)
    FAIL: What are the events of photosynthesis?
size 1200, overlap 180 (15%)
    all gold facts kept together

```

2 Section 16: Chunk Overlap Experiment

```

[32]: # Overlap grid
base_sizes = [800, 1000, 1200]
overlap_pcts = [0, 0.10, 0.15, 0.20]
doc_chars = sum(len(p.page_content) for pages in docs_clean.values() for p in
↪pages)

rows = []
for size in base_sizes:
    for pct in overlap_pcts:
        ov = round(size * pct)
        by_doc = {n: chunk_document(p, make_splitter(size, ov)) for n, p in
↪docs_clean.items()}
        allc = [c for ch in by_doc.values() for c in ch]
        rows.append({
            "size": size, "overlap": ov, "chunks": len(allc),
            "gold_%": gold_score(by_doc),
            "paras_whole_%": round(np.mean([paragraph_integrity(by_doc[n], n)
↪for n in docs_clean])),
            "redundancy_x": round(sum(len(c.page_content) for c in allc) /
↪doc_chars, 2),
        })

overlap_table = pd.DataFrame(rows)
overlap_table

```

[32]:	size	overlap	chunks	gold_%	paras_whole_%	redundancy_x
0	800	0	161	100	94	1.00
1	800	80	161	100	94	1.00
2	800	120	167	100	93	1.03
3	800	160	171	100	93	1.06
4	1000	0	130	95	96	1.00
5	1000	100	131	98	96	1.01
6	1000	150	133	100	96	1.03
7	1000	200	135	100	96	1.05
8	1200	0	104	95	98	1.00
9	1200	120	104	98	98	1.02
10	1200	180	106	100	98	1.04
11	1200	240	106	100	98	1.06

2.1 Section 17: Final Decision

Decision: `chunk_size = 1000`, `chunk_overlap = 150` (15% of the size).

Provisional. Experiment 2 retrieval results favour size 800 on Hit@5 and size 1200 on Hit@1, so the chunk size is re-decided in Experiment 2.

```
[33]: # Decision (from Section 15 + Section 16)
FINAL_SIZE = 1000
FINAL_OVERLAP_PCT = 0.15
FINAL_OVERLAP = round(FINAL_SIZE * FINAL_OVERLAP_PCT)

print(f"DECISION: chunk_size={FINAL_SIZE}, chunk_overlap={FINAL_OVERLAP}␣
↳({round(FINAL_OVERLAP_PCT * 100)}% of size)\n")
print("Size table (12% overlap):")
print(size_table.to_string(index=False))
print(f"\nOverlap table (size {FINAL_SIZE}):")
print(overlap_table[overlap_table["size"] == FINAL_SIZE].to_string(index=False))
print("\nEffect of cleaning (size 500, chunks without end punctuation):")
print(cleaning_table[["doc", "version", "no_end_punct_%"]].
↳to_string(index=False))

final_by_doc = {n: chunk_document(p, make_splitter(FINAL_SIZE, FINAL_OVERLAP))␣
↳for n, p in docs_clean.items()}
final_gold = gold_score(final_by_doc)
print(f"\nGold at the final setting: {final_gold}%")
print("DECISION CHECK:", "PASS" if final_gold == 100 else "FAIL - see which␣
↳questions fail with gold_failures(final_by_doc)")
```

DECISION: `chunk_size=1000`, `chunk_overlap=150` (15% of size)

Size table (12% overlap):

size	overlap	chunks	avg_len	max_len	tiny_<100	gold_%	paras_whole_%
avg_tokens		max_tokens					

500	60	257	365	546	0	93	80
85	213						
800	96	164	580	892	0	100	93
135	272						
1000	120	131	726	1066	0	100	96
168	340						
1200	144	105	914	1194	0	98	98
211	364						

Overlap table (size 1000):

size	overlap	chunks	gold_%	paras_whole_%	redundancy_x
1000	0	130	95	96	1.00
1000	100	131	98	96	1.01
1000	150	133	100	96	1.03
1000	200	135	100	96	1.05

Effect of cleaning (size 500, chunks without end punctuation):

doc	version	no_end_punct_%
doc1_life_pr	raw	79
doc1_life_pr	clean + old splitter	24
doc1_life_pr	clean + new chunking	5
doc2_electri	raw	72
doc2_electri	clean + old splitter	30
doc2_electri	clean + new chunking	6

Gold at the final setting: 100%

DECISION CHECK: PASS

2.2 Section 18: Save the Outputs for Experiment 2

Run the two cells below. They use FINAL_SIZE and FINAL_OVERLAP_PCT from Section 17. chunks_800 / 1000 / 1200.jsonl use the same overlap percentage, so Experiment 2 compares sizes fairly.

```
[34]: # Export chunks (sizes 800 / 1000 / 1200 + the final choice)
def tag_chunk_type(text, section=None):
    sec = (section or "").lower()
    if sec.startswith("exercises"):
        return "exercise"
    if sec.startswith("what you have learnt"):
        return "summary"
    lines = [l for l in text.split("\n") if l.strip()]
    q_lines = sum(bool(re.match(r"^\d{1,2}\.\s*[A-Z]", l)) for l in lines)
    if lines and q_lines / len(lines) >= 0.5 and "?" in text:
        return "exercise"
    if re.search(r"(?m)^\s*Activity\s?\d", text):
        return "activity"
    if re.search(r"(?m)^\s*Example\s\d", text):
        return "example"
    return "content"

def export_chunks(size, overlap, filename):
```

```

    by_doc = {n: chunk_document(p, make_splitter(size, overlap)) for n, p in docs_clean.items()}
    k = 0
    with open(PROCESSED_DIR / filename, "w", encoding="utf-8") as f:
        for name, chunks in by_doc.items():
            for i, c in enumerate(chunks):
                rec = {
                    "chunk_id": f"{Path(name).stem}_{size}_{i:04d}",
                    "text": c.page_content,
                    "source": name,
                    "page": c.metadata["page"] + 1,          # 1-based page
                    "section": c.metadata.get("section"),
                    "chunk_type": tag_chunk_type(c.page_content, c.metadata.get("section")), # CHANGED
                    "char_count": len(c.page_content),
                }
                f.write(json.dumps(rec, ensure_ascii=False) + "\n")
                k += 1
    return k, gold_score(by_doc)

PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

exported = {}
for size in [800, 1000, 1200]:
    ov = round(size * FINAL_OVERLAP_PCT)
    n, g = export_chunks(size, ov, f"chunks_{size}.jsonl")
    exported[f"chunks_{size}.jsonl"] = {"chunk_size": size, "chunk_overlap": ov, "chunks": n, "gold_score_percent": g}
    print(f"chunks_{size}.jsonl (overlap {ov}) -> {n} chunks | gold {g}%")

n_chunks, gold_final = export_chunks(FINAL_SIZE, FINAL_OVERLAP, "chunks.jsonl")
print(f"chunks.jsonl (size {FINAL_SIZE}, overlap {FINAL_OVERLAP}) -> {n_chunks} chunks | gold {gold_final}%")
print("GOLD CHECK:", "PASS" if gold_final == 100 else f"FAIL ({gold_final}% - see the failures cell in Section 16)")

config = {
    "chunk_version": "v2",
    "splitter": "RecursiveCharacterTextSplitter",
    "chunk_size": FINAL_SIZE,
    "chunk_overlap": FINAL_OVERLAP,
    "chunk_overlap_pct": FINAL_OVERLAP_PCT,
    "separators": SEPARATORS,
    "keep_separator": "end",
    "pages_joined_before_split": True,

```

```

    "split_by_section": True,
    "tiny_chunk_min_len": 100,
    "page_numbering": "1-based",
    "documents": list(docs_clean.keys()),
    "total_chunks": n_chunks,
    "gold_score_percent": gold_final,
    "gold_questions": len(gold),
    "gold_version": "v2",
    "exported_files": exported,
}
with open(PROCESSED_DIR / "chunking_config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

# Save the cleaned pages too, so later notebooks do not have to re-clean
with open(PROCESSED_DIR / "clean_pages.jsonl", "w", encoding="utf-8") as f:
    for name, pages in docs_clean.items():
        for p in pages:
            f.write(json.dumps({"source": name, "page": p.metadata["page"] + 1,
↪ "text": p.page_content},
                               ensure_ascii=False) + "\n")

```

```

chunks_800.jsonl (overlap 120) -> 167 chunks | gold 100%
chunks_1000.jsonl (overlap 150) -> 133 chunks | gold 100%
chunks_1200.jsonl (overlap 180) -> 106 chunks | gold 100%
chunks.jsonl (size 1000, overlap 150) -> 133 chunks | gold 100%
GOLD CHECK: PASS

```

2.3 Section 19: Not Solved Yet

Problems this notebook does not fix. They carry over to later work.

1. Formulas lose fractions and subscripts (for example 1 volt = 1 joule / 1 coulomb, and the parallel-resistance formula). PyPDF cannot read them. Next step: try PyMuPDF, or read formula pages with a vision model.
2. Figure captions are extracted in the middle of sentences. The caption starts a new paragraph and the rest of the sentence gets glued to the caption text. About 6 chunks end mid-sentence because of this.
3. Drop-cap and spacing problems stay: headings like “5.3 RESPIR ATION”, words like “fr om” and “nor mal”, and small gaps such as “HEA TING”.
4. (Fixed) The page icon “?” and the in-text “QUESTIONS” box headings are removed; “What you have learnt” and “EXERCISES” now start their own sections.
5. An Activity (steps plus questions) can still be split across chunks, and a list can be separated from its intro sentence.
6. Figures and diagrams are ignored. Only their captions survive.
7. Only two clean digital PDFs were tested. No scanned pages, slides, Word files or handwritten notes. The “<200 chars per page” check in Section 5 is the first signal for sending a page to OCR.
8. The gold set has 42 development questions plus a separate held-out test set. It can tell good

settings from bad ones, but not tiny differences.

9. Tables become number soup and attract top-1 results.
10. Identifiers like Activity 11.1 and 11.2 look alike to the models.
11. The cleaning rules and the section titles (SECTION_TITLES) are written for these two NCERT chapters. Other uploads need a generic version (for example PyMuPDF headings and running-header detection).

2.4 Next: Experiment 3 (reranking)

Experiment 2 loads the exported chunks, embeds them and builds the top-20 candidates. Experiment 3 reranks those candidates and runs the held-out test.

```
[35]: # Read the export back and check it
records = [json.loads(line) for line in open(PROCESSED_DIR / "chunks.jsonl",
      ↪encoding="utf-8")]
df = pd.DataFrame(records)

print("Total chunks:", len(df))
print(df.groupby("source")["char_count"].describe().round(0))
print("\nchunk types:\n", df["chunk_type"].value_counts().to_string())

checks = {
    "start with punctuation": int(df["text"].str[0].isin(list(".?;,:")).sum())
    ↪== 0,
    "start lowercase": int(df["text"].str[0].str.islower().sum()) == 0,
    "tiny (<100 chars)": int((df["char_count"] < 100).sum()) == 0,
    "duplicate chunk_ids": int(df["chunk_id"].duplicated().sum()) == 0,
    "chunks without section": int(df["section"].isna().sum()) == 0,
    "empty text": int((df["text"].str.strip() == "").sum()) == 0,
    "gold file exists": gold_path.exists(),
    "config file exists": (PROCESSED_DIR / "chunking_config.json").exists(),
}
print()
for name, ok in checks.items():
    print(f"  {'PASS' if ok else 'FAIL'}  {name}")
print("\nEXPORT CHECK:", "PASS" if all(checks.values()) else "FAIL")
print("\nFirst chunk:\n", df.iloc[0]["text"][:400])
```

Total chunks: 133

	count	mean	std	min	25%	50%	75%	\
source								
doc1_life_processes.pdf	71.0	739.0	231.0	171.0	614.0	803.0	922.0	
doc2_electricity.pdf	62.0	715.0	272.0	104.0	513.0	846.0	939.0	
		max						
source								
doc1_life_processes.pdf	998.0							
doc2_electricity.pdf	1066.0							

chunk types:

	chunk_type
content	82
exercise	18
activity	13
example	11
summary	9

PASS start with punctuation
PASS start lowercase
PASS tiny (<100 chars)
PASS duplicate chunk_ids
PASS chunks without section
PASS empty text
PASS gold file exists
PASS config file exists

EXPORT CHECK: PASS

First chunk:

How do we tell the difference between what is alive and what is not alive? If we see a dog running, or a cow chewing cud, or a man shouting loudly on the street, we know that these are living beings. What if the dog or the cow or the man were asleep? We would still think that they were alive, but how did we know that? We see them breathing, and we know that they are alive. What about plants? How d

```
[36]: print("Section labels per document:")
for src, g in df.groupby("source"):
    print(" ", src)
    for s in g["section"].drop_duplicates():
        print("    ", s)

print("\nchunk types:\n", df["chunk_type"].value_counts().to_string())

numbered = df[df["section"].str.match(r"^\d+\.\d+\s")]
leaky = numbered[numbered["section"].str.contains(r"[a-z)")]
print("\nnumbered labels with body text left in them:", leaky["section"].
      ↪unique(), "(want 0)")
print("chunks ending with a stray '?' + QUESTIONS  :", int(df["text"].str.
      ↪contains(r"\?\s*\n+\s*QUESTIONS\s*$").sum()), "(want 0)")
```

Section labels per document:

doc1_life_processes.pdf
Chapter opening
5.1 WHAT ARE LIFE PROCESSES?
5.2 NUTRITION

```

5.3 RESPIRATION
5.4 TRANSPORTATION
5.5 EXCRETION
What you have learnt
EXERCISES
doc2_electricity.pdf
Chapter opening
11.1 ELECTRIC CURRENT AND CIRCUIT
11.2 ELECTRIC POTENTIAL AND POTENTIAL DIFFERENCE
11.3 CIRCUIT DIAGRAM
11.4 OHM'S LAW
11.5 FACTORS ON WHICH THE RESISTANCE OF A CONDUCTOR DEPENDS
11.6 RESISTANCE OF A SYSTEM OF RESISTORS
11.7 HEATING EFFECT OF ELECTRIC CURRENT
11.8 ELECTRIC POWER
What you have learnt
EXERCISES

chunk types:
chunk_type
content      82
exercise     18
activity     13
example      11
summary      9

numbered labels with body text left in them: 0 (want 0)
chunks ending with a stray '?' + QUESTIONS : 0 (want 0)

```

2.5 Step 2: new diagnostic

```

[37]: EXPECTED = {
    "doc1_life_processes.pdf": [f"5.{i}" for i in range(1, 6)],
    "doc2_electricity.pdf":    [f"11.{i}" for i in range(1, 9)],
}

print("Section sequence (in document order):")
all_ok = True
for src, g in df.groupby("source"):
    print(" ", src)
    runs = (g["section"] != g["section"].shift()).cumsum()
    for _, run in g.groupby(runs):
        print(f"      {run['section'].iloc[0][:44]:44s} chunks {len(run):2d} |
↳ pages {run['page'].min()}-{run['page'].max()}")
        found = set(g["section"].str.extract(r"^\d+\.\d+")[0].dropna())
        missing = [n for n in EXPECTED[src] if n not in found]
        print("      missing main sections:", missing if missing else "none")

```

```

    all_ok = all_ok and not missing
print("\nSECTION CHECK:", "PASS" if all_ok else "FAIL - see the context lines_
↳below")

if not all_ok:
    print("\nWhere the missing section numbers appear in the joined text:")
    for name, pages in docs_clean.items():
        full, _ = join_pages(pages)
        g = df[df["source"] == name]
        found = set(g["section"].str.extract(r"^\d+\.\d+")[0].dropna())
        for num in [n for n in EXPECTED[name] if n not in found]:
            hits = [m.start() for m in re.finditer(r"(?![\d.])" + re.
↳escape(num) + r"(?![\d.])", full)]
            print(f"    {name[:12]} {num}: {len(hits)} mentions")
            for pos in hits[:4]:
                print("        ", repr(full[max(0, pos - 25): pos + 70]))

bad_labels = []
for src, g in df.groupby("source"):
    for s in g["section"].drop_duplicates():
        num = re.match(r"^\d+\.\d+", s)
        if num and s != SECTION_TITLES[src].get(num.group(1)):
            bad_labels.append(s)
print("\nLABEL CHECK:", "PASS" if not bad_labels else f"FAIL - {bad_labels}")

dup_head = [bool(re.search(r"(\d+\.\d+)+s[A-Z][A-Z ,'\--]{1,?})\s?\1", t)) for_
↳t in df["text"]]
print("chunks with a repeated heading inside the text:", sum(dup_head), "(want_
↳0)")
print("\nSTEP CHECK:", "PASS" if all_ok and not bad_labels and not_
↳any(dup_head) else "FAIL")

```

Section sequence (in document order):

doc1_life_processes.pdf	
Chapter opening	chunks 3 pages 1-1
5.1 WHAT ARE LIFE PROCESSES?	chunks 5 pages 1-3
5.2 NUTRITION	chunks 18 pages 3-8
5.3 RESPIRATION	chunks 13 pages 9-12
5.4 TRANSPORTATION	chunks 19 pages 13-18
5.5 EXCRETION	chunks 6 pages 18-20
What you have learnt	chunks 5 pages 20-21
EXERCISES	chunks 2 pages 21-21
missing main sections: none	
doc2_electricity.pdf	
Chapter opening	chunks 1 pages 1-1
11.1 ELECTRIC CURRENT AND CIRCUIT	chunks 4 pages 1-2
11.2 ELECTRIC POTENTIAL AND POTENTIAL DIFFER	chunks 4 pages 3-3

11.3 CIRCUIT DIAGRAM	chunks	1		pages	4-4
11.4 OHM'S LAW	chunks	7		pages	5-7
11.5 FACTORS ON WHICH THE RESISTANCE OF A CO	chunks	12		pages	7-11
11.6 RESISTANCE OF A SYSTEM OF RESISTORS	chunks	12		pages	11-17
11.7 HEATING EFFECT OF ELECTRIC CURRENT	chunks	10		pages	18-20
11.8 ELECTRIC POWER	chunks	2		pages	21-21
What you have learnt	chunks	4		pages	22-22
EXERCISES	chunks	5		pages	23-24
missing main sections: none					

SECTION CHECK: PASS

LABEL CHECK: PASS

chunks with a repeated heading inside the text: 0 (want 0)

STEP CHECK: PASS

Chosen setup: hybrid retriever (bge-base + BM25), top 20 candidates, reranked by bge-reranker-base on the plain chunk text, blended with the retrieval rank by RRF, plus the Activity/Example number boost. Chunk size 1200 (overlap 180).

Why: it passed the fixed rule (Hit@20 $\geq$ 96, reads all 20 candidates, worse than the retriever on at most 2 questions) and is best by composite or within the 2-point tie band, with the fewest “worse on”. On the 42 development questions: Hit@1 79, Hit@3 95, Hit@5 98, Hit@10 100, Hit@20 100, MRR 0.874 (retriever alone at size 1200: Hit@1 69, Hit@3 83, Hit@5 90, MRR 0.773). Better on 9 questions, worse on 0. Size 1000 had a slightly higher composite (90.6) but made 1 question worse; the two are tied within the band, so the rule prefers 1200.

Why not the alternatives: pool 10 cannot see ranks 11 to 20, and the first held-out set had two questions with evidence at rank 12. Plain reranking without RRF was worse than the retriever on 6 to 9 questions; RRF reduced that to 0 to 2. The ID boost added about 5 points of Hit@1.

Known weaknesses (from the failure analysis): exercise and summary chunks outrank the evidence for some definition and list questions; look-alike Activities (for example Activity 11.4 above the series-resistance evidence); the events of photosynthesis are spread over neighbouring chunks; tables and formulas lost by the PDF reader (Experiment 4).

Targets for the held-out set (fixed now, before running): Hit@5 of at least 85%, Hit@20 of at least 96%, reranker worse than the retriever on at most 2 of the 24 questions, and Hit@1 not lower than the retriever alone. One question is worth about 4 points.

```
[44]: # [Cell 41] Held-out test set v2 (24 fresh questions): build, fill the evidence
      ↪phrases from the book text, check, save.
      # Write ONCE. Never tune on it. Never look at retrieval results while writing
      ↪it,
      # and never edit it after Experiment 3 has run it.
      # Nothing in this cell uses the retriever. Phrases come from the cleaned book
      ↪text only.
```

```

ALLOWED_TYPES = {"definition", "formula", "fact", "list", "process",
    ↪ "reasoning", "activity", "example"}
P, B = "doc2_electricity.pdf", "doc1_life_processes.pdf"

# ----- helpers
    ↪ (book text only)
dev_text = " ".join(g["question"] for g in gold).lower()

def full_doc_text(doc):
    return "\n".join(p.page_content for p in docs_clean[doc])

def id_context(doc, kind, num, n=1200):
    t = full_doc_text(doc)
    m = re.search(rf"(?m)^[kind]\s?(re.escape(num))\b", t)
    return t[m.start(): m.start() + n] if m else ""

def free_ids(doc, kind):
    """Activity/Example numbers found in the text that no dev question names
    ↪ and whose text no dev question uses as evidence."""
    t = full_doc_text(doc)
    nums = sorted({m.group(1) for m in re.finditer(rf"(?m)^[kind]\s?(d+\.
    ↪ d+)", t)},
        key=lambda s: tuple(int(x) for x in s.split(".")))
    dev_phr = [p.lower() for g in gold if g["doc"] == doc for p in
    ↪ g["must_contain"] if len(p) >= 20]
    out = []
    for n_ in nums:
        if f"{kind.lower()} {n_}" in dev_text:
            continue
        ctx = id_context(doc, kind, n_).lower()
        if any(p in ctx for p in dev_phr):
            continue
        out.append(n_)
    return out

def candidates(doc, keywords, n=4):
    """Lines/sentences of the cleaned text that contain the most keywords."""
    text = full_doc_text(doc)
    parts = [s.strip() for s in re.split(r"\n+|(?<=[?!])\s+", text) if len(s.
    ↪ strip()) > 30]
    scored = sorted(((sum(k.lower() in s.lower() for k in keywords), s) for s
    ↪ in parts), key=lambda x: -x[0])
    return [s for sc, s in scored[:n] if sc > 0]

def close_together(text, phrases, span=700):
    """Every phrase must occur within `span` characters of the first one."""

```

```

t = text.lower()
first = [m.start() for m in re.finditer(re.escape(phrases[0].lower()), t)]
return any(all(any(abs(m.start() - i) < span for m in re.finditer(re.
↪escape(p.lower()), t)) for p in phrases[1:]) for i in first)

def safe_runs(sentence):
    """Runs of consecutive plain words (no Omega, degree signs, curly quotes,
↪dashes)."""
    runs, cur = [], []
    for w in sentence.split():
        if re.fullmatch(r"[A-Za-z0-9,.;:()/'\~]+", w):
            cur.append(w)
        else:
            if cur:
                runs.append(cur)
            cur = []
    if cur:
        runs.append(cur)
    return runs

def sentence_phrases(sentence):
    runs = sorted([r for r in safe_runs(sentence) if len(r) >= 4], key=len,
↪reverse=True)
    if not runs:
        return None
    out = [" ".join(runs[0][:9])]
    if len(runs) > 1:
        out.append(" ".join(runs[1][:8]))
    return out

def concept_sentences(g, n=8):
    out = []
    for s in candidates(g["doc"], g["keywords"], n=n):
        s = s.strip().rstrip("- ").strip()
        if s.endswith("?") or re.match(r"^(Figure|Fig\.|Table)\b", s) or len(s)
↪< 40:
            continue
        out.append(s)
    return out

def id_sentences(g, n=6):
    kind, num = g["id"]
    ctx = id_context(g["doc"], kind, num, n=900)
    out = []
    for s in re.split(r"\n+|(?<=[.?!])\s+", ctx):
        s = re.sub(r"^(Activity|Example)\s?\d+\.\d+\s*", "", s.strip()).
↪rstrip("- ").strip()

```

```

        if len(s) < 30 or s.endswith("?") or re.match(r"^(Figure|Fig\.|
↪|Table)\b", s):
            continue
        out.append(s)
    return out[:n]

# ----- pick
↪Activities / Examples the dev set does not use
pa, pe, ba = free_ids(P, "Activity"), free_ids(P, "Example"), free_ids(B,
↪"Activity")
print("Free Physics Activities:", pa, "| free Physics Examples:", pe, "| free
↪Biology Activities:", ba)
assert len(pa) >= 2 and len(pe) >= 2 and len(ba) >= 3, "not enough unused
↪Activities/Examples - send me this printout"
pick = lambda xs, k: [xs[round(i * (len(xs) - 1) / max(k - 1, 1))]] for i in
↪range(k)]
a1, a2 = pick(pa, 2); e1, e2 = pick(pe, 2); b1, b2, b3 = pick(ba, 3)

# ----- the 24
↪questions
test_gold = [
    # ----- Physics (12): 8 concept + 2 Activity + 2 Example
    ↪-----
    {"doc": P, "type": "fact", "question": "Which instrument is used to
↪measure electric current and how is it connected in a circuit?",
    "keywords": ["ammeter", "series", "current"], "must_contain": []},
    {"doc": P, "type": "fact", "question": "In which direction does
↪conventional current flow compared with the movement of electrons?",
    "keywords": ["conventional", "direction", "electrons"], "must_contain":
↪[]},
    {"doc": P, "type": "definition", "question": "When is the resistance of a
↪conductor said to be 1 ohm?",
    "keywords": ["1 ohm", "ohm", "ampere"], "must_contain": []},
    {"doc": P, "type": "reasoning", "question": "Why are alloys used in
↪heating devices such as electric irons and toasters instead of pure metals?",
    "keywords": ["alloy", "heating", "resistivity"], "must_contain": []},
    {"doc": P, "type": "reasoning", "question": "Why is tungsten used for the
↪filament of an electric bulb?",
    "keywords": ["tungsten", "filament", "melting"], "must_contain": []},
    {"doc": P, "type": "formula", "question": "What does Joule's law of
↪heating say about the heat produced in a resistor?",
    "keywords": ["Joule", "law of heating", "proportional"], "must_contain":
↪[]},
    {"doc": P, "type": "reasoning", "question": "What happens to the current
↪in a circuit when the resistance is increased but the potential difference
↪stays the same?",

```

```

    "keywords": ["inversely proportional", "resistance", "current"],
    ↪ "must_contain": [],
    {"doc": P, "type": "list",      "question": "What are some everyday uses
    ↪ of the heating effect of electric current?",
    "keywords": ["heating effect", "heater", "iron", "bulb"], "must_contain":
    ↪ []},
    {"doc": P, "type": "activity",  "question": f"What is done and what do we
    ↪ observe in Activity {a1}?",
    "id": ("Activity", a1), "keywords": [], "must_contain": []},
    {"doc": P, "type": "activity",  "question": f"What does Activity {a2} show
    ↪ us?",
    "id": ("Activity", a2), "keywords": [], "must_contain": []},
    {"doc": P, "type": "example",   "question": f"What values are given in
    ↪ Example {e1} and what has to be found?",
    "id": ("Example", e1), "keywords": [], "must_contain": []},
    {"doc": P, "type": "example",   "question": f"How is the answer to Example
    ↪ {e2} worked out?",
    "id": ("Example", e2), "keywords": [], "must_contain": []},
    # ----- Biology (12): 9 concept + 3 Activity -----
    {"doc": B, "type": "definition", "question": "What is the difference
    ↪ between autotrophic and heterotrophic nutrition?",
    "keywords": ["autotrophic", "heterotrophic"], "must_contain": []},
    {"doc": B, "type": "process",     "question": "Why is bile juice important
    ↪ for the digestion of fats?",
    "keywords": ["bile", "fats", "emulsif", "liver"], "must_contain": []},
    {"doc": B, "type": "process",     "question": "How do villi in the small
    ↪ intestine help in absorbing digested food?",
    "keywords": ["villi", "absorption", "surface area"], "must_contain": []},
    {"doc": B, "type": "process",     "question": "How do the ribs and the
    ↪ diaphragm move when we breathe in?",
    "keywords": ["diaphragm", "ribs", "breathe in", "lungs"], "must_contain":
    ↪ []},
    {"doc": B, "type": "fact",        "question": "What is the function of the
    ↪ alveoli in the lungs?",
    "keywords": ["alveoli", "exchange", "blood"], "must_contain": []},
    {"doc": B, "type": "fact",        "question": "What does haemoglobin do in
    ↪ the blood?",
    "keywords": ["haemoglobin", "oxygen"], "must_contain": []},
    {"doc": B, "type": "reasoning",   "question": "Why is double circulation
    ↪ necessary in human beings?",
    "keywords": ["double circulation", "oxygenated", "warm-blooded",
    ↪ "energy"], "must_contain": []},
    {"doc": B, "type": "definition", "question": "What is blood pressure and
    ↪ what are systolic and diastolic pressure?",
    "keywords": ["blood pressure", "systolic", "diastolic"], "must_contain":
    ↪ []},

```

```

    {"doc": B, "type": "reasoning", "question": "What is transpiration and why
↳is it useful for the plant?",
    "keywords": ["transpiration", "absorption of water", "temperature"],
    ↳"must_contain": []},
    {"doc": B, "type": "activity", "question": f"What is done and what is
↳observed in Activity {b1}?",
    "id": ("Activity", b1), "keywords": [], "must_contain": []},
    {"doc": B, "type": "activity", "question": f"What is the aim of Activity
↳{b2} and what does it show?",
    "id": ("Activity", b2), "keywords": [], "must_contain": []},
    {"doc": B, "type": "activity", "question": f"What do we conclude from
↳Activity {b3}?",
    "id": ("Activity", b3), "keywords": [], "must_contain": []},
]
assert len(test_gold) == 24, "expected 24 questions"

# ----- hand-written
↳phrases (these always win over auto-fill)
# To replace a weak automatic pick, add the question number here with 1-2 EXACT
↳phrases from the cleaned text, e.g.
# 13: ["first exact phrase", "second exact phrase"],
PHRASES = {
    1: ["An instrument called ammeter measures electric current in a circuit",
↳"It is always connected in series in a circuit"],
    2: ["direction of electric current is taken as opposite to the direction of
↳the flow of electrons", "which are negative charges"],
    3: ["potential difference across the two ends of a conductor is 1 V", "the
↳current through it is 1 A"],
    4: ["the resistivity of an alloy is generally higher", "than that of its
↳constituent metals"],
    5: ["high melting point such as tungsten", "is used for making bulb
↳filaments"],
    7: ["current through a resistor is inversely proportional to its
↳resistance"],
    8: ["The electric laundry iron, electric toaster, electric oven, electric
↳kettle", "are some of the familiar devices based on Joule"],
}
for i, ph in PHRASES.items():
    test_gold[i - 1]["must_contain"] = ph

# ----- the SAME
↳chunks the export cell writes (15% overlap)
EXPORT_CHUNKS = {s: {n: chunk_document(p, make_splitter(s, round(s *
↳FINAL_OVERLAP_PCT))) for n, p in docs_clean.items()}}
    for s in (800, 1000, 1200)}

```

```

def in_one_chunk(g, size):
    return any(all(p.lower() in c.page_content.lower() for p in
    ↪g["must_contain"]) for c in EXPORT_CHUNKS[size][g["doc"]])

# ----- auto-fill
↪the questions that still have no phrases
def try_fill(g, sentences):
    """First sentence whose phrases are in the cleaned text AND inside one
    ↪chunk at sizes 800, 1000 and 1200."""
    for s in sentences:
        ph = sentence_phrases(s)
        if not ph or not all(p.lower() in full_text[g["doc"]] for p in ph):
            continue
        g["must_contain"] = ph
        if all(in_one_chunk(g, size) for size in (800, 1000, 1200)):
            return s
        g["must_contain"] = []
    return None

for i, g in enumerate(test_gold, 1):
    if g["must_contain"]:
        continue
    s = try_fill(g, id_sentences(g) if g.get("id") else concept_sentences(g))
    if s:
        g["_src"] = s

# ----- read these
↪before running Experiment 3
print("\nEVIDENCE PHRASES (A = filled automatically from the book text, H =
↪written by hand)")
for i, g in enumerate(test_gold, 1):
    tag = "A" if g.get("_src") else "H"
    print(f"{i:>2} [{tag}] {g['question'][:95]}")
    if g["must_contain"]:
        for p in g["must_contain"]:
            print("    -", p)
        if tag == "A":
            print("    from:", g["_src"][:170])
    else:
        print("    -> NO usable sentence found: write this one in PHRASES")

# ----- checks
dev_q = {g["question"] for g in gold}
problems = []
if len({g["question"] for g in test_gold}) != len(test_gold):
    problems.append(("all", "duplicate questions in the test set", ""))
for i, g in enumerate(test_gold, 1):

```

```

    if not g["must_contain"]:
        problems.append((i, "no phrases", "")); continue
    text = full_text[g["doc"]]
    miss = [p for p in g["must_contain"] if p.lower() not in text]
    if miss:
        problems.append((i,
↪ "phrase not in cleaned text", miss)); continue
    if g["type"] not in ALLOWED_TYPES:
        problems.append((i, "bad_
↪ type", g["type"]))
    if g["question"] in dev_q:
        problems.append((i, "same_
↪ as a dev question", ""))
    if not close_together(text, g["must_contain"]):
        problems.append((i,
↪ "phrases are more than 700 chars apart", ""))
    bad_sizes = [s for s in (800, 1000, 1200) if not in_one_chunk(g, s)]
    if bad_sizes:
        problems.append((i,
↪ f"evidence split across exported chunks at sizes {bad_sizes}", ""))
print()
for p in problems:
    print("PROBLEM:", p)
print("TEST SET CHECK:", "PASS" if not problems else "FAIL", "|",
↪ len(test_gold), "questions")
print(pd.Series([g["doc"][:12] for g in test_gold]).value_counts().to_string())
print("questions naming an Activity/Example:", sum(1 for g in test_gold if g.
↪ get("id")))
print("filled automatically:", sum(1 for g in test_gold if g.get("_src")), "|
↪ written by hand:", sum(1 for g in test_gold if not g.get("_src")))

# ----- save_
↪ (refuses to overwrite once Experiment 3 has run the set)
HELD_RESULT = EVAL_DIR.parent / "results" / "heldout_v2_results.json"
test_path = EVAL_DIR / "gold_questions_test.json"
if problems:
    print("\nNot saved: fix the PROBLEM lines first.")
elif HELD_RESULT.exists():
    print("\nNot saved: Experiment 3 already ran this set (", HELD_RESULT.name,
↪ "). Never edit it after that.")
else:
    out = [{k: v for k, v in g.items() if k not in ("keywords", "id", "_src")}
↪ for g in test_gold]
    with open(test_path, "w", encoding="utf-8") as f:
        json.dump(out, f, indent=2, ensure_ascii=False)
    with open(test_path, encoding="utf-8") as f:
        back = json.load(f)
    ok = len(back) == 24 and all(q["must_contain"] for q in back)
    print("\nSaved", len(back), "questions to", test_path)
    print("SAVE CHECK:", "PASS" if ok else "FAIL")

```

Free Physics Activities: ['11.1', '11.5', '11.6'] | free Physics Examples:

['11.2', '11.3', '11.4', '11.5', '11.6', '11.7', '11.8', '11.10', '11.11', '11.12'] | free Biology Activities: ['5.2', '5.4', '5.5', '5.6', '5.7']

EVIDENCE PHRASES (A = filled automatically from the book text, H = written by hand)

- 1 [H] Which instrument is used to measure electric current and how is it connected in a circuit?
 - An instrument called ammeter measures electric current in a circuit
 - It is always connected in series in a circuit
- 2 [H] In which direction does conventional current flow compared with the movement of electrons?
 - direction of electric current is taken as opposite to the direction of the flow of electrons
 - which are negative charges
- 3 [H] When is the resistance of a conductor said to be 1 ohm?
 - potential difference across the two ends of a conductor is 1 V
 - the current through it is 1 A
- 4 [H] Why are alloys used in heating devices such as electric irons and toasters instead of pure meta
 - the resistivity of an alloy is generally higher
 - than that of its constituent metals
- 5 [H] Why is tungsten used for the filament of an electric bulb?
 - high melting point such as tungsten
 - is used for making bulb filaments
- 6 [A] What does Joule's law of heating say about the heat produced in a resistor?
 - $I^2 R t$ (11.21) This is known as
 - from: (11.5)], we get $H = I^2 R t$ (11.21) This is known as Joule's law of heating.
- 7 [H] What happens to the current in a circuit when the resistance is increased but the potential dif
 - current through a resistor is inversely proportional to its resistance
- 8 [H] What are some everyday uses of the heating effect of electric current?
 - The electric laundry iron, electric toaster, electric oven, electric kettle
 - are some of the familiar devices based on Joule
- 9 [A] What is done and what do we observe in Activity 11.1?
 - Set up a circuit as shown in Fig.
 - from: Set up a circuit as shown in Fig.
- 10 [A] What does Activity 11.6 show us?
 - Make a parallel combination, XY, of three resistors having
 - from: Make a parallel combination, XY, of three resistors having resistances R_1 , R_2 , and R_3 , respectively.
- 11 [A] What values are given in Example 11.2 and what has to be found?
 - Solution The amount of charge Q , that flows between
 - 12 V) is 2 C.
 - from: Solution The amount of charge Q , that flows between two points at potential difference V ($= 12 \text{ V}$) is 2 C.

- 12 [A] How is the answer to Example 11.12 worked out?
- An electric bulb is connected to a 220 V
from: An electric bulb is connected to a 220 V generator.
- 13 [A] What is the difference between autotrophic and heterotrophic nutrition?
- Heterotrophic organisms include animals and fungi.
from: Heterotrophic organisms include animals and fungi.
- 14 [A] Why is bile juice important for the digestion of fats?
- Bile juice from the liver accomplishes this in addition
from: Bile juice from the liver accomplishes this in addition to acting on fats.
- 15 [A] How do villi in the small intestine help in absorbing digested food?
- The inner lining of the small intestine has numerous
from: The inner lining of the small intestine has numerous finger-like projections called villi which increase the surface area for absorption.
- 16 [A] How do the ribs and the diaphragm move when we breathe in?
- As we have seen in earlier years, when we
from: As we have seen in earlier years, when we breathe in, we lift our ribs and flatten our diaphragm, and the chest cavity becomes larger as a result.
- 17 [A] What is the function of the alveoli in the lungs?
- The alveoli provide a surface where the exchange of
from: The alveoli provide a surface where the exchange of gases can take place.
- 18 [A] What does haemoglobin do in the blood?
- In human beings, the respiratory pigment is haemoglobin which
from: In human beings, the respiratory pigment is haemoglobin which has a very high affinity for oxygen.
- 19 [A] Why is double circulation necessary in human beings?
- Since these maintenance processes are needed to prevent damage
from: Since these maintenance processes are needed to prevent damage and break-down, energy is needed for them.
- 20 [A] What is blood pressure and what are systolic and diastolic pressure?
- The pressure of blood inside the artery during ventricular
from: The pressure of blood inside the artery during ventricular systole (contraction) is called systolic pressure and pressure in artery during ventricular diastole (relaxation)
- 21 [A] What is transpiration and why is it useful for the plant?
- This is useful in animals that have high energy
from: This is useful in animals that have high energy needs, such as birds and mammals, which constantly use energy to maintain their body temperature.
- 22 [A] What is done and what is observed in Activity 5.2?
- So far, we have talked about how autotrophs
from: So far, we have talked about how autotrophs meet their energy requirements.
- 23 [A] What is the aim of Activity 5.5 and what does it show?
- Take some fruit juice or sugar solution and add
from: Take some fruit juice or sugar solution and add some yeast to this.

24 [A] What do we conclude from Activity 5.7?
- Visit a health centre in your locality and find
from: Visit a health centre in your locality and find out what is the
normal range of haemoglobin content in human beings.

TEST SET CHECK: PASS | 24 questions

doc2_electri 12

doc1_life_pr 12

questions naming an Activity/Example: 7

filled automatically: 17 | written by hand: 7

Saved 24 questions to

c:\Users\ASUS\Desktop\VisualMind\data\eval\gold_questions_test.json

SAVE CHECK: PASS

```
[45]: import json
t = json.load(open(EVAL_DIR / "gold_questions_test.json", encoding="utf-8"))
print(len(t), "questions | first:", t[0]["question"][:60])
```

24 questions | first: Which instrument is used to measure electric current and
how

[]: